

Conferencia 5: cotas mínimas

Diseño y Análisis de Algoritmos · Ciencia de la Computación · MatCom · 2026

En la conferencia 2 cambiamos una vez la pregunta, de "cuánto cuesta este algoritmo" a "cuánto cuesta cualquiera", y la respondimos con una reducción desde un teorema que citamos sin demostrar. Hoy aprendemos a demostrar cotas mínimas nosotros mismos. Vamos a trabajar en el modelo de comparaciones, que es el más simple donde todo se puede hacer con papel y lápiz, y a ver dos herramientas, cada una con su límite. Al final la cota mínima va a cambiar de papel: además de decir cuánto cuesta un problema, va a servir para detectar algoritmos que están mal.

1. De este algoritmo a cualquier algoritmo

Para decir algo sobre **todos** los algoritmos hay que decir primero qué es un algoritmo. En esta clase, un algoritmo del **modelo de comparaciones** solo puede acceder a los datos preguntando si $a_i < a_j$. Puede hacer todo el cómputo que quiera con los índices y con las respuestas, pero sobre los valores solo sabe lo que le dijeron las comparaciones. Su costo es el número de preguntas. Ordenamiento por inserción, mergesort, heapsort, la búsqueda binaria y el máximo por recorrido son todos algoritmos de este modelo.

Recuerda la conferencia 2: una cota mínima vale en su modelo, y fuera de él puede romperse. La sección 9 vuelve sobre esto con el ordenamiento.

Hay tres maneras de demostrar una cota mínima, y las tres tienen nombre:

- **Información:** contar cuántas respuestas distintas tiene que poder dar el algoritmo.
- **Adversario:** imaginar un oponente que contesta las preguntas para que el algoritmo aprenda lo menos posible.
- **Reducción:** mostrar que un algoritmo rápido para este problema daría uno rápido para otro problema que ya sabemos difícil. Es lo que hicimos con Ben-Or en la conferencia 2, y es la herramienta del Tema 3 entero.

La cota $\Omega(n \log n)$ para ordenar por comparaciones se suele enunciar sin demostración. Hoy se demuestra entera, paso por paso, y después vemos hasta dónde llega esa técnica.

2. Árboles de decisión

Fijemos n y un algoritmo de comparaciones. La primera pregunta que hace no depende de la entrada, porque todavía no sabe nada. La segunda depende solo de la respuesta a la primera, y así sucesivamente. Todas las ejecuciones posibles del algoritmo forman entonces un **árbol de decisión**: cada nodo interno es una comparación, con una rama por respuesta, y cada hoja es una salida. El costo en el peor caso es la **altura** del árbol.

Cómo se le ocurre a uno. El árbol no hay que dibujarlo a mano. Corre el algoritmo sobre todas las entradas posibles, anota las preguntas de cada ejecución y pega las ejecuciones por donde coinciden.

```
def ordenar_por_insercion(A, menor):
    A = list(A)
    for i in range(1, len(A)):
        j = i
        while j > 0 and menor(A[j], A[j - 1]):
            A[j], A[j - 1] = A[j - 1], A[j]
            j -= 1
    return A
```

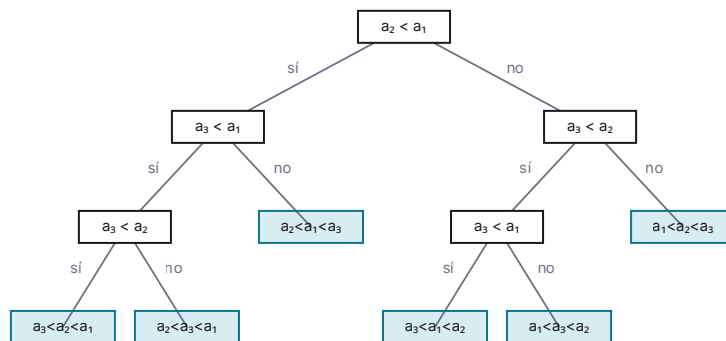


Figura 1. El árbol de decisión de ordenamiento por inserción para $n = 3$, obtenido corriendo el algoritmo sobre las seis permutaciones. Seis hojas, una por orden posible, y altura 3.

La figura 1 es el árbol de ordenamiento por inserción para $n = 3$. Tiene seis hojas, una por cada orden posible de tres elementos, y altura 3. Ningún algoritmo puede ordenar tres elementos con dos comparaciones, y la sección siguiente dice por qué.

3. El argumento de información

La cota de ordenamiento sale en tres pasos, y cada paso es un lema corto:

1. Un árbol de decisión bajo no puede tener muchas hojas.
2. Un algoritmo que ordena necesita muchas hojas: una por cada orden posible de la entrada.
3. Cuánto es "muchas": el valor de $\log_2 n!$.

Juntando los tres sale que el árbol no puede ser bajo, que es lo mismo que decir que el algoritmo hace muchas comparaciones en el peor caso.

Paso 1: un árbol bajo tiene pocas hojas

Lema 1. Un árbol binario de altura h tiene a lo sumo 2^h hojas.

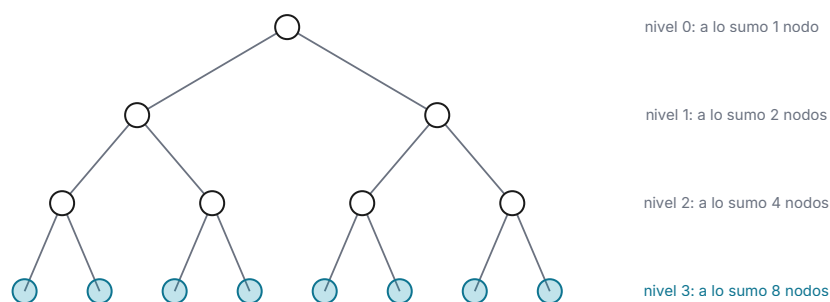


Figura 2. Un árbol binario de altura 3 con todos sus niveles llenos. Cada nivel tiene a lo sumo el doble de nodos que el anterior, así que el último tiene a lo sumo $2^3 = 8$ hojas.

La idea está en la figura 2: cada nodo tiene a lo sumo dos hijos, así que cada nivel tiene a lo sumo el doble de nodos que el anterior. El nivel 0 tiene un nodo, el nivel 1 a lo sumo dos, el nivel 2 a lo sumo cuatro, y el nivel h a lo sumo 2^h .

Demostración. Por inducción sobre h .

- Si $h = 0$, el árbol es una sola hoja, y $1 = 2^0$.

- Si $h > 0$, la raíz tiene a lo sumo dos subárboles. Cada uno tiene altura a lo sumo $h - 1$, así que, por hipótesis de inducción, cada uno tiene a lo sumo 2^{h-1} hojas. Entre los dos, a lo sumo $2 \cdot 2^{h-1} = 2^h$. $\square$

Lo vamos a usar al revés: **si un árbol tiene al menos L hojas, su altura es al menos $\log_2 L$** , porque con altura menor tendría menos de $2^{\log_2 L} = L$ hojas.

Paso 2: ordenar necesita $n!$ hojas

Lema 2. El árbol de decisión de un algoritmo de comparaciones que ordena n elementos tiene al menos $n!$ hojas.

Antes de demostrarlo, veámoslo pasar. La tabla siguiente corre el ordenamiento por inserción de la sección 2 sobre las seis entradas posibles con tres elementos distintos, y anota las respuestas que recibe cada una y la hoja donde termina.

ENTRADA (a_1, a_2, a_3)	RESPUESTAS, EN ORDEN	HOJA: LA SALIDA
1, 2, 3	$a_2 < a_1$: no; $a_3 < a_2$: no	$a_1 < a_2 < a_3$
1, 3, 2	$a_2 < a_1$: no; $a_3 < a_2$: sí; $a_3 < a_1$: no	$a_1 < a_3 < a_2$
2, 1, 3	$a_2 < a_1$: sí; $a_3 < a_1$: no	$a_2 < a_1 < a_3$
2, 3, 1	$a_2 < a_1$: no; $a_3 < a_2$: sí; $a_3 < a_1$: sí	$a_3 < a_1 < a_2$
3, 1, 2	$a_2 < a_1$: sí; $a_3 < a_1$: sí; $a_3 < a_2$: no	$a_2 < a_3 < a_1$
3, 2, 1	$a_2 < a_1$: sí; $a_3 < a_1$: sí; $a_3 < a_2$: sí	$a_3 < a_2 < a_1$

Seis entradas, **6 hojas distintas**.

Cada entrada termina en una hoja distinta, y no es casualidad de este algoritmo. Es obligatorio para cualquier algoritmo que ordene, y la razón tiene dos partes.

Demostración.

- **La salida depende solo de la hoja.** El algoritmo nunca ve los valores; solo ve las respuestas. Por eso su salida tiene que estar escrita en términos de posiciones, como en la última columna de la tabla: "primero el de la posición 2, después el de la 1, después el de la 3". Y dos entradas que reciben las mismas respuestas recorren el mismo camino, llegan a la misma hoja y producen la misma salida.
- **Dos órdenes distintos necesitan salidas distintas.** Si dos entradas tienen órdenes distintos, hay dos posiciones i y j tales que en una $a_i < a_j$ y en la otra $a_i > a_j$. La salida correcta pone i antes que j en la primera y j antes que i en la segunda.

Si dos entradas con órdenes distintos llegaran a la misma hoja, recibirían la misma salida por la primera parte, y por la segunda esa salida estaría mal para una de las dos. Así que cada uno de los $n!$ órdenes tiene su propia hoja. $\square$

Paso 3: cuánto vale $\log_2 n!$

Lema 3. $\log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2}$, y con más cuidado, $\log_2 n! \geq n \log_2 n - 1,443 n$.

La primera cota es elemental. Con $n = 8$:

$$8! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot \underbrace{5 \cdot 6 \cdot 7 \cdot 8}_{\text{cada uno} \geq 4} \geq 4^4.$$

En general, la mitad más grande de los factores de $n!$ son todos al menos $n/2$, así que $n! \geq (n/2)^{n/2}$. Tomando logaritmo, $\log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2}$, que ya es $\Omega(n \log n)$.

La segunda afina la constante comparando la suma con una integral.

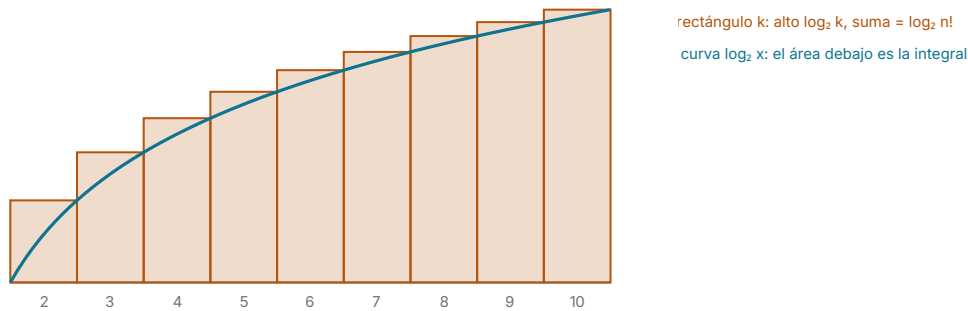


Figura 3. $\log_2 10!$ es la suma de las áreas de los rectángulos: el rectángulo k tiene base 1 y alto $\log_2 k$. Como $\log_2$ es creciente, cada rectángulo queda por encima de la curva en su intervalo, así que la suma es al menos el área bajo la curva, que es la integral.

En la [figura 3](#), el rectángulo número k ocupa el intervalo $[k-1, k]$ y tiene alto $\log_2 k$. Como $\log_2$ crece, en ese intervalo la curva nunca pasa de $\log_2 k$, así que el rectángulo cubre el área bajo la curva:

$\log_2 k \geq \int_{k-1}^k \log_2 x \, dx$. Sumando de $k=2$ a n ,

$$\log_2 n! = \sum_{k=2}^n \log_2 k \geq \int_1^n \log_2 x \, dx = n \log_2 n - (n-1) \log_2 e \geq n \log_2 n - 1,443 n,$$

porque una primitiva de $\log_2 x$ es $x \log_2 x - x \log_2 e$, y $\log_2 e \approx 1,4427$. $\square$

El teorema

Teorema. Todo algoritmo de ordenamiento por comparaciones hace, en el peor caso, al menos $\lceil \log_2 n! \rceil \geq n \log_2 n - 1,443 n$ comparaciones.

Demostración. El peor caso es la altura del árbol. Por el lema 2 el árbol tiene al menos $n!$ hojas, y por el lema 1 su altura es al menos $\log_2 n!$. La altura es un entero, así que es al menos $\lceil \log_2 n! \rceil$. El lema 3 da el valor. $\square$

Con números:

n	$n!$	$\log_2 n!$	COMPARACIONES NECESARIAS
3	6	2,58	3
4	24	4,58	5
5	120	6,91	7
10	3 628 800	21,79	22

Para $n=3$ hacen falta 3, y el árbol de la [figura 1](#), que tiene altura 3, es óptimo.

¿Está cerca de lo que hacen los algoritmos reales? Contemos el peor caso exacto de mergesort, recorriendo **todas** las permutaciones para $n \leq 8$.

```
def mezclar(I, D, cuenta, registro=None):
    R, i, j = [], 0, 0
    while i < len(I) and j < len(D):
        cuenta[0] += 1
        if registro is not None:
            registro.append((I[i], D[j]))
        if I[i] <= D[j]:
            R.append(I[i])
            i += 1
        else:
            R.append(D[j])
            j += 1
    return R + I[i:] + D[j:]

def mergesort(A, cuenta):
    if len(A) <= 1:
        return list(A)
    m = len(A) // 2
    return mezclar(mergesort(A[:m], cuenta), mergesort(A[m:], cuenta), cuenta)

peor_mergesort, informacion = [], []
print(f"{'n':>3} {'mergesort, peor caso':>21} {'[log2 n!']:>10}")
for n in range(2, 9):
    peor = 0
    for P in permutations(range(n)):
        cuenta = [0]
        mergesort(list(P), cuenta)
        peor = max(peor, cuenta[0])
    peor_mergesort.append(peor)
    informacion.append(math.ceil(math.log2(math.factorial(n))))
print(f"{'n':>3} {'peor:>21} {'informacion[-1]:>10}")
```

n	mergesort, peor caso	[log ₂ n!]
2	1	1
3	3	3
4	5	5
5	8	7
6	11	10
7	14	13
8	17	16

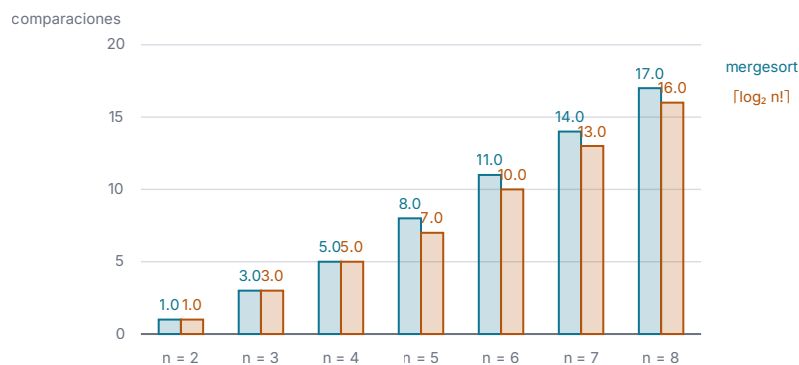


Figura 4. Peor caso exacto de mergesort, recorriendo todas las permutaciones, contra la cota de información. La diferencia es de una comparación a partir de $n = 5$.

En este rango la diferencia es de una sola comparación. Para n grande crece, pero despacio: cuando n es potencia de 2, el peor caso de mergesort es exactamente $n \log_2 n - n + 1$, y la cota es $n \log_2 n - 1,443 n$, así que se separan en unas $0,44 n$ comparaciones. Coinciden en el término principal, que es lo que dice que mergesort es óptimo salvo por el término lineal. El ejercicio 8 pregunta qué pasaría con un algoritmo que prometiera menos.

La búsqueda binaria también es óptima

El mismo argumento sirve para buscar. Tenemos un arreglo ordenado $a_1 < a_2 < \dots < a_n$ y un valor x , y el algoritmo compara x con elementos del arreglo. Cada comparación responde $<, =$ o $>$.

Aquí hay una trampa. Con tres respuestas posibles, el árbol tiene tres ramas por nodo, y el lema 1 adaptado a tres ramas solo daría $\log_3$. Para conseguir $\log_2$ miramos solo los casos en que x **no está** en el arreglo.

Los huecos. Si x no está, cae en uno de los $n + 1$ huecos entre elementos: antes de a_1 , entre a_1 y a_2 , ..., o después de a_n . En esos casos ninguna comparación puede responder $=$, así que solo se usan dos ramas.

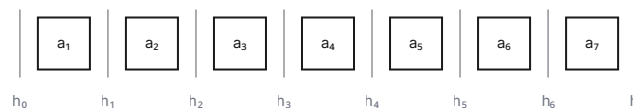


Figura 5. Un arreglo ordenado de siete elementos y sus ocho huecos, h_0 a h_7 . Si x no está en el arreglo, está en exactamente uno de ellos.

Lema 4. Si x_1 y x_2 caen en huecos distintos, un algoritmo correcto los lleva a hojas distintas.

Demostración. Supongamos que x_1 cae en un hueco a la izquierda del de x_2 , y que los dos terminan en la misma hoja.

- Entre sus huecos hay al menos un elemento a_i , con $x_1 < a_i < x_2$.
- El algoritmo no comparó con ese a_i . Si lo hubiera hecho, x_1 habría recibido " $<$ " y x_2 habría recibido " $>$ ", y habrían seguido caminos distintos.
- Ahora tomemos $x = a_i$. En cada comparación que el algoritmo hace, con un elemento a la izquierda de los dos huecos o a la derecha, a_i recibe la misma respuesta que x_1 y x_2 . Así que $x = a_i$ recorre el mismo camino y llega a la misma hoja.
- En esa hoja el algoritmo dice "no está", porque es la hoja de x_1 . Pero a_i sí está. El algoritmo se equivoca. $\square$

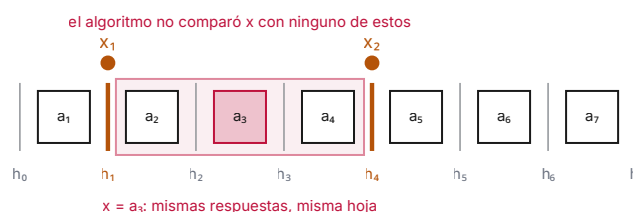


Figura 6. La demostración del lema 4 con x_1 en el hueco h_1 y x_2 en el h_4 . Si llegan a la misma hoja, el algoritmo no comparó con a_2 , a_3 ni a_4 . Entonces $x = a_3$ recibe las mismas respuestas y el algoritmo dice que no está.

Consecuencia. Los $n + 1$ huecos necesitan $n + 1$ hojas distintas, en la parte del árbol que solo usa las ramas $<$ y $>$, que es un árbol binario. Por el lema 1, hacen falta al menos $\lceil \log_2(n + 1) \rceil$ comparaciones.

La búsqueda binaria las alcanza. Cada comparación parte por la mitad el rango donde todavía puede estar x , así que después de $\lceil \log_2(n+1) \rceil$ comparaciones queda un solo hueco. El árbol de abajo sale de correr la búsqueda binaria sobre un valor de cada hueco.

```
def busqueda_binaria(A, x, comparar):
    """Devuelve ("está", posición) o ("hueco", número de hueco)."""
    lo, hi = 0, len(A)
    while lo < hi:
        m = (lo + hi) // 2
        r = comparar(x, A[m])
        if r == 0:
            return ("está", m)
        if r < 0:
            hi = m
        else:
            lo = m + 1
    return ("hueco", lo)
```

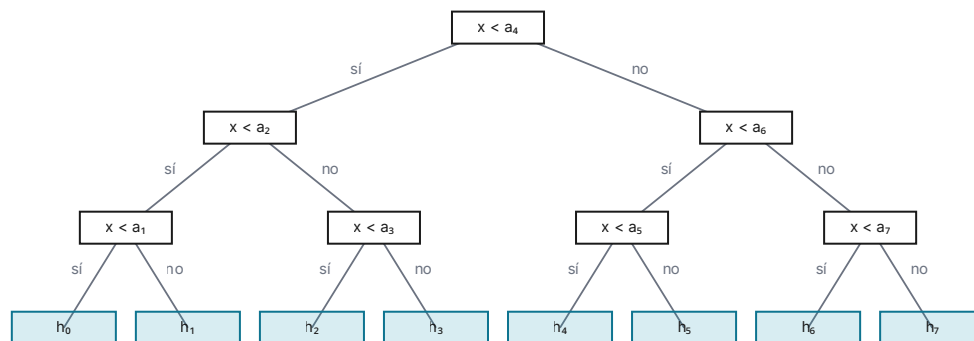


Figura 7. El árbol de decisión de la búsqueda binaria sobre siete elementos, obtenido corriéndola con un valor de cada hueco. Ocho hojas, una por hueco, y altura $3 = \lceil \log_2 8 \rceil$: justo la cota del lema 4.

Con $n = 7$ hay ocho huecos, y la búsqueda binaria los separa con tres comparaciones, que es exactamente $\lceil \log_2 8 \rceil$. Es óptima.

4. Cuándo la información no alcanza

El argumento de información cuenta salidas. Cuando la salida es pequeña pero el trabajo para justificarla es grande, se queda corto.

- **El máximo.** Hay n salidas posibles, así que la información da $\log_2 n$. Pero todo el mundo sabe que el máximo cuesta $n - 1$ comparaciones. El árbol no lo ve, porque cuenta respuestas, no el trabajo de descartar a los demás candidatos.
- **Mezclar dos listas ordenadas de largo n .** Hay $\binom{2n}{n}$ maneras de intercalarlas, y el algoritmo de mezcla usa $2n - 1$ comparaciones.
- **El segundo mayor.** Hay $n(n - 1)$ salidas si contamos el par (mayor, segundo), y el torneo de la sección 7 usa $n + \lceil \log_2 n \rceil - 2$.

```
print(f"{'n':>5} {'mezclar: info':>14} {'mezclar: real':>14} "
      f"{'2º mayor: info':>15} {'2º mayor: real':>15}")
for n in [4, 16, 64, 256, 1024]:
    info_mezcla = math.log2(math.comb(2 * n, n))
    info_segundo = math.log2(n * (n - 1))
    real_segundo = n + math.ceil(math.log2(n)) - 2
    print(f"{'n':>5} {'info_mezcla':>14.1f} {'2 * n - 1':>14} "
          f"{'info_segundo':>15.1f} {'real_segundo':>15}")
```

n	mezclar: info	mezclar: real	2° mayor: info	2° mayor: real
4	6.1	7	3.6	4
16	29.2	31	7.9	18
64	124.2	127	12.0	68
256	507.2	511	16.0	262
1024	2042.2	2047	20.0	1032

Para mezclar, la información queda a una distancia de $\frac{1}{2}\log_2 n$ más una constante: cerca, pero no llega. Para el segundo mayor queda en $2\log_2 n$ contra algo lineal: no sirve para nada. En los dos casos el costo está en **verificar** la respuesta, no en **elegirla**, y para eso hace falta otra herramienta.

5. El adversario

La idea es cambiar de punto de vista. En lugar de fijar una entrada y mirar qué hace el algoritmo, imaginamos que el algoritmo juega contra un **adversario** que contesta las comparaciones. El adversario no se compromete con ninguna entrada al principio: solo tiene que contestar de forma **consistente**, es decir, de manera que en todo momento exista al menos una entrada que daría todas las respuestas dadas hasta ahora.

La cota sale así. Si el algoritmo termina y todavía quedan dos entradas consistentes con todas las respuestas pero con salidas correctas distintas, el algoritmo se equivocó en una de las dos. Entonces un algoritmo correcto tiene que seguir preguntando hasta que eso no pase, y la cota mínima es cuántas preguntas puede forzar el adversario.

El máximo, $n - 1$. Aquí el adversario ni siquiera necesita estrategia. Si al terminar hay un elemento $x \neq m$ que nunca perdió una comparación, el adversario puede subir el valor de x por encima de todos sin contradecir ninguna respuesta, y entonces el máximo sería x , no m . Así que los $n - 1$ elementos distintos del máximo tienen que perder al menos una vez. Cada comparación tiene un solo perdedor, así que hacen falta al menos $n - 1$ comparaciones.

El adversario es la herramienta de contradicción de la conferencia 3 aplicada a todas las ejecuciones a la vez: si el algoritmo pregunta poco, existe una entrada donde se equivoca, y el adversario es la receta para construirla.

6. Mezclar: $2n - 1$

Teorema. Mezclar dos listas ordenadas de largo n requiere $2n - 1$ comparaciones en el peor caso, y el algoritmo de mezcla las alcanza.

Demostración. La mezcla usa a lo sumo $2n - 1$: cada comparación saca un elemento a la salida, y el último sale sin comparar. Para la cota mínima, el adversario contesta como si la entrada fuera la **intercalada perfecta**

$$a_1 < b_1 < a_2 < b_2 < \dots < a_n < b_n.$$

Esa cadena tiene $2n - 1$ pares de vecinos, y cada par tiene un elemento de cada lista. Supongamos que un algoritmo termina sin haber comparado un par de vecinos, por ejemplo a_i con b_i . Intercambiamos los valores de a_i y b_i . Las dos listas siguen ordenadas, porque a_i y b_i son vecinos en la cadena y nada queda entre ellos. Todas las comparaciones que el algoritmo hizo dan la misma respuesta, porque ninguna involucraba a los dos a la vez y el orden con el resto no cambió. Pero la salida correcta cambió: ahora b_i va antes que a_i . El algoritmo da la misma salida para las dos entradas, así que se equivoca en una. Por tanto compara los $2n - 1$ pares. $\square$

```
for n in [4, 16, 64]:
    A, B = list(range(0, 2 * n, 2)), list(range(1, 2 * n, 2)) # intercaladas
    cuenta, registro = [0], []
    mezclar(A, B, cuenta, registro)
    print(f"n = {n:>3}: {cuenta[0]} comparaciones, 2n - 1 = {2 * n - 1}")
```

```
n = 4: 7 comparaciones, 2n - 1 = 7
n = 16: 31 comparaciones, 2n - 1 = 31
n = 64: 127 comparaciones, 2n - 1 = 127
```

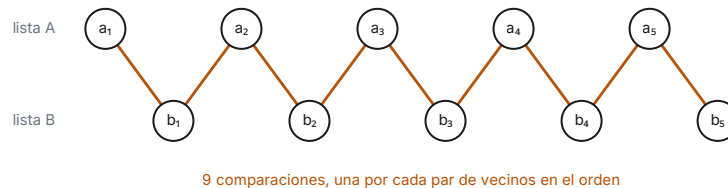


Figura 8. La entrada intercalada con $n = 5$. Las líneas son las comparaciones que la mezcla hizo de verdad al correr sobre ella: exactamente los nueve pares de vecinos de la cadena, sin sobrar ninguno.

La [figura 8](#) muestra las comparaciones que la mezcla hace sobre la entrada del adversario: son exactamente los pares de vecinos. Esta cota es también el problema 8-6 de CLRS, que la plantea como ejercicio guiado.

7. El segundo mayor: $n + \lceil \log_2 n \rceil - 2$

El segundo mayor por fuerza bruta es fácil: encuentra el máximo con $n - 1$ comparaciones, quítalo, y encuentra el máximo de lo que queda con $n - 2$ más. Son $2n - 3$. ¿Se puede mejor?

Cómo se le ocurre a uno. El segundo mayor perdió contra el máximo en algún momento, porque contra cualquier otro habría ganado. Así que no hace falta buscarlo entre todos: basta buscarlo entre los que perdieron **contra el máximo**. El problema es que el máximo por recorrido le puede ganar a $n - 1$ elementos. Hay que organizar las comparaciones para que el máximo le gane a pocos.

El torneo. Los elementos juegan por parejas; los ganadores pasan a la ronda siguiente, y así hasta que queda uno, que es el máximo. Son $n - 1$ comparaciones, porque cada una elimina a un jugador. El máximo jugó una vez por ronda, y hay $\lceil \log_2 n \rceil$ rondas. Su segundo es el mayor de sus rivales, que cuesta $\lceil \log_2 n \rceil - 1$ comparaciones más. En total, $n + \lceil \log_2 n \rceil - 2$.

```
def torneo(A, saltar_primera=False):
    """Devuelve (segundo mayor, comparaciones). Cada jugador lleva la lista de a
    quiénes venció, en el orden de las rondas."""
    comparaciones = 0
    jugadores = [(x, []) for x in A]
    while len(jugadores) > 1:
        ronda = []
        for i in range(0, len(jugadores) - 1, 2):
            (a, va), (b, vb) = jugadores[i], jugadores[i + 1]
            comparaciones += 1
            ronda.append((a, va + [b]) if a > b else (b, vb + [a]))
        if len(jugadores) % 2:
            ronda.append(jugadores[-1]) # pasa de ronda sin jugar
        jugadores = ronda
    maximo, vencidos = jugadores[0]
    candidatos = vencidos[1:] if saltar_primera else vencidos
    segundo = candidatos[0]
    for c in candidatos[1:]:
```

```

    comparaciones += 1
    if c > segundo:
        segundo = c
    return segundo, comparaciones

random.seed(7)
print(f"{'n':>5} {'peor caso medido':>17} {'n + [log2 n] - 2':>18}")
for n in [8, 64, 1024]:
    peor = 0
    for _ in range(2000):
        A = random.sample(range(10 * n), n)
        segundo, c = torneo(A)
        assert segundo == sorted(A)[-2]
        peor = max(peor, c)
    print(f"{'n':>5} {'peor':>17} {'n + math.ceil(math.log2(n)) - 2':>18}")

```

n	peor caso medido	$n + \lceil \log_2 n \rceil - 2$
8	9	9
64	68	68
1024	1032	1032

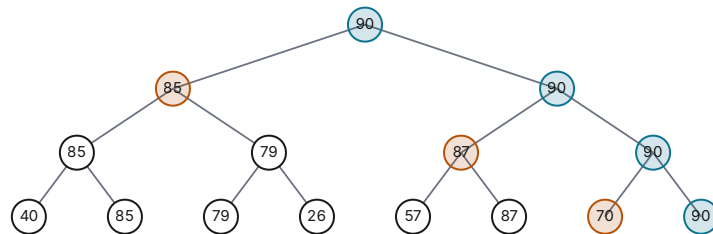


Figura 9. El torneo sobre 40, 85, 79, 26, 57, 87, 70, 90. En azul, los partidos del máximo; en naranja, sus tres rivales. El segundo mayor, 87, es uno de ellos, y solo hay que compararlos entre sí.

El peor caso medido coincide con la fórmula. Ahora la pregunta de la clase: ¿se puede con menos?

Teorema. Todo algoritmo de comparaciones que encuentra el segundo mayor de n elementos hace, en el peor caso, al menos $n + \lceil \log_2 n \rceil - 2$ comparaciones.

Demostración. Va en dos pasos: primero, qué tiene que saber el algoritmo al terminar; después, cómo el adversario lo obliga a pagar por saberlo.

Qué tiene que saber. Sea m el máximo y s el segundo, según la entrada que el adversario termine fijando. Al terminar:

- Todo elemento distinto de m perdió alguna comparación. Si dos elementos u y v nunca perdieron, el adversario puede ponerlos a los dos por encima de todos, en cualquiera de los dos órdenes, y el segundo mayor sería v o u según cuál elija.
- De los elementos que perdieron **solo contra** m , hay exactamente uno, que es s . Si hubiera otro, x , los dos solo tendrían a m por encima según las respuestas, y el adversario podría poner cualquiera de los dos inmediatamente debajo de m .

Sea k el número de elementos que fueron comparados con m . Todos perdieron contra m , y todos salvo s perdieron además contra algún otro elemento. Contemos derrotas: cada comparación produce exactamente una. Los $n - 1$ elementos distintos de m pierden al menos una vez, y los $k - 1$ rivales de m distintos de s pierden al menos dos. En total hay al menos $(n - 1) + (k - 1)$ derrotas, y otras tantas comparaciones.

Cómo obliga el adversario a que k sea grande. Cada elemento empieza con **peso**

1. Cuando el algoritmo compara x con y :

- Si los dos tienen peso positivo, gana el de mayor peso (en empate, cualquiera), y el ganador absorbe el peso del perdedor, que queda en 0.
- Si solo uno tiene peso positivo, gana ese, y los pesos no cambian.
- Si los dos tienen peso 0, contesta de forma consistente con lo que ya dijo.

Los elementos con peso positivo son exactamente los que nunca perdieron, y la suma de los pesos es siempre n . Las respuestas son consistentes: un elemento con peso positivo nunca perdió, así que declararlo ganador no puede cerrar un ciclo. Al terminar, solo m no perdió nunca, así que su peso es n . El peso de m solo sube cuando le gana a un elemento de peso positivo, que tiene a lo sumo su mismo peso, así que en cada una de esas comparaciones su peso a lo sumo se duplica. Para llegar de 1 a n necesita al menos $\lceil \log_2 n \rceil$ de ellas. Entonces $k \geq \lceil \log_2 n \rceil$, y el total es al menos $n + \lceil \log_2 n \rceil - 2$. $\square$

La [figura 9](#) muestra por qué el torneo es óptimo: organiza las comparaciones para que el máximo le gane exactamente a $\lceil \log_2 n \rceil$ rivales, que es lo mínimo que el adversario permite.

8. La cota mínima como detector de errores

En las conferencias 2, 3 y 4 plantamos un error en un algoritmo, y la demostración de correctitud o de costo decía dónde buscarlo. Aquí el error lo detecta la cota mínima.

Mira de nuevo el torneo. El rival de la primera ronda del máximo perdió en el primer partido. Parece un candidato flojo, y saltárselo ahorra una comparación. Es el parámetro `saltar_primera` del código de arriba. La versión con atajo usa $n + \lceil \log_2 n \rceil - 3$ comparaciones: **una menos que la cota mínima**.

Sin mirar el código, sabemos que está mal: ningún algoritmo correcto puede hacer menos que la cota. Y la demostración dice en qué entrada falla. El atajo nunca compara al rival de la primera ronda con ningún otro rival del máximo, así que ese rival perdió solo contra el máximo, y queda más de un elemento en esa situación. El adversario pone a ese rival como segundo mayor. Construyamos la entrada, y midamos con qué frecuencia lo detectan pruebas al azar.

```
n = 16
A = [n - 1, n - 2] + random.sample(range(n - 2), n - 2)  # el 2º mayor, en la 1ª ronda
print("entrada construida:", A)
print("torneo:", torneo(A)[0], " atajo:", torneo(A, saltar_primera=True)[0],
      " verdad:", sorted(A)[-2])
print()

random.seed(11)
print(f"{'n':>5} {'pruebas':>8} {'fallos del atajo':>17} {'1/(n-1)':>9}")
for n, pruebas in [(8, 20_000), (64, 20_000), (1024, 4_000)]:
    fallos = 0
    for _ in range(pruebas):
        A = random.sample(range(10 * n), n)
        fallos += torneo(A, saltar_primera=True)[0] != sorted(A)[-2]
    print(f"{'n':>5} {'pruebas':>8} {100 * fallos / pruebas:>16.2f}% "
          f"{100 / (n - 1):>8.2f}%")
```

```
entrada construida: [15, 14, 9, 1, 13, 0, 7, 4, 3, 12, 5, 11, 8, 6, 10, 2]
torneo: 14 atajo: 13 verdad: 14
```

n	pruebas	fallos del atajo	1/(n-1)
8	20000	14.31%	14.29%

64	20000	1.65%	1.59%
1024	4000	0.12%	0.10%

El atajo falla exactamente cuando el segundo mayor es el rival de la primera ronda del máximo, que en una entrada al azar pasa con probabilidad $1/(n-1)$. Con 1024 elementos, una de cada mil pruebas. Una batería de pruebas al azar con arreglos grandes no lo ve casi nunca. La cota mínima lo vio sin correr nada.

La lección cierra la serie de errores plantados del tema. **Una cota mínima es también una especificación:** un algoritmo que dice hacer menos trabajo que una cota demostrada está mal, aunque pase todas las pruebas. Y si parece estar bien, lo que hay que revisar es en qué modelo trabaja, que es la sección siguiente.

9. Qué modelo

Ordenar cuesta $\Omega(n \log n)$ comparaciones. Pero si los valores son enteros en un rango $[0, k)$ conocido, se puede ordenar sin comparar nada.

```
def ordenar_por_conteo(A, k):
    cuenta = [0] * k
    for x in A:
        cuenta[x] += 1
    salida = []
    for v in range(k):
        salida.extend([v] * cuenta[v])
    return salida

random.seed(4)
n, k = 100_000, 1000
A = [random.randrange(k) for _ in range(n)]
cuenta = [0]
assert ordenar_por_conteo(A, k) == mergesort(A, cuenta) == sorted(A)
print(f"mergesort: {cuenta[0]:,} comparaciones")
print(f"conteo:    0 comparaciones y {n + k:,} pasos (n + k)")
print(f"[log2 n!]: {math.ceil(math.lgamma(n + 1) / math.log(2)):,}")
```

```
mergesort: 1,536,234 comparaciones
conteo:    0 comparaciones y 101,000 pasos (n + k)
[log2 n!]: 1,516,705
```

El ordenamiento por conteo cuesta $O(n+k)$, por debajo de $n \log n$ cuando $k = O(n)$. No contradice el teorema de la sección 3, porque no es un algoritmo de comparaciones: usa los valores como **índices** de un arreglo. Es el mismo truco que la rejilla de la conferencia 2, que usaba la parte entera como dirección para romper la cota de Ben-Or.

La regla es la de la conferencia 2, y ya la vimos aplicarse muchas veces: una cota mínima viene con su modelo pegado. La cota de ordenamiento vale en comparaciones. La de Ben-Or, en árboles de decisión algebraicos. La de Fredman y Saks de la conferencia 4, en sondeo de celdas. Antes de citar una cota, hay que saber cuál es su modelo y qué deja fuera.

10. Cierre del Tema 1

Con esta conferencia termina el Tema 1. Después de cinco conferencias tenemos las herramientas para las cinco preguntas del ciclo: medir en un modelo (conferencia 1), recorrer el ciclo entero (conferencia 2), demostrar correctitud (conferencia 3), analizar el costo de secuencias de operaciones (conferencia 4) y

acotar por abajo (esta). El documento de cierre del tema junta todo en unas pocas páginas y trae ejercicios que mezclan las cinco conferencias.

El Tema 2 cambia de eje. Las clases dejan de organizarse por pregunta y pasan a organizarse por técnica de diseño: divide y vencerás, golosos, programación dinámica, flujos. Las cinco preguntas siguen ahí, y siguen siendo las que se evalúan.

11. Resumen

- Una cota mínima es una afirmación sobre todos los algoritmos de un modelo. En el modelo de comparaciones, el costo es el número de preguntas $a_i < a_j$.
- Un algoritmo de comparaciones es un árbol de decisión, y su peor caso es la altura.
- **Información:** con L salidas posibles, la altura es al menos $\log_2 L$. Ordenar cuesta al menos $\lceil \log_2 n! \rceil \geq n \log_2 n - 1.443 n$, y mergesort está a distancia lineal de esa cota.
- La información cuenta salidas, no trabajo. Para el máximo da $\log_2 n$ en vez de $n - 1$.
- **Adversario:** un oponente que contesta de forma consistente y fuerza preguntas. Da $n - 1$ para el máximo, $2n - 1$ para mezclar y $n + \lceil \log_2 n \rceil - 2$ para el segundo mayor, y las tres cotas se alcanzan.
- La cota del segundo mayor sale de contar derrotas y de un adversario de pesos que obliga al máximo a jugar $\lceil \log_2 n \rceil$ veces. El torneo lo logra.
- Un algoritmo que hace menos que una cota demostrada está mal. El torneo que se salta un rival falla con probabilidad $1/(n - 1)$, y la cota lo detecta sin pruebas.
- El ordenamiento por conteo no contradice la cota: no compara. Toda cota viene con su modelo.

Ejercicios

1. **Máximo y mínimo a la vez.** Diseña un algoritmo que los encuentre con $\lceil 3n/2 \rceil - 2$ comparaciones, y demuestra con un adversario que no se puede con menos. (Pista: clasifica cada elemento según si ya ganó, ya perdió, las dos cosas o ninguna.)
2. Demuestra que decidir si un arreglo está ordenado requiere $n - 1$ comparaciones.
3. Mezclar una lista de largo m con otra de largo n : da la cota de información en función de m y n , y un algoritmo que la alcance cuando $m = 1$.
4. Da un algoritmo que encuentre la mediana de 5 elementos con 6 comparaciones. (Que con 5 no alcanza también es cierto, pero la demostración es un análisis de casos largo.)
5. **Distinción de elementos en el modelo de comparaciones.** Demuestra $\Omega(n \log n)$ con un árbol de decisión. ¿Por qué no basta contar salidas, si solo hay dos ("sí" o "no")?
6. El tercer mayor: extiende el torneo para encontrarlo y cuenta cuántas comparaciones usa en el peor caso. ¿Qué dice la cota de información?
7. Radix sort ordena n enteros de b bits en $O(nb/\log n)$ pasos usando dígitos de $\log_2 n$ bits. ¿En qué modelo? ¿Contradice la sección 3?
8. Alguien afirma tener un algoritmo de ordenamiento por comparaciones que hace a lo sumo $n \log_2 n - 2n$ comparaciones para todo n grande. ¿Puede existir? Justifica con la sección 3.